Übungen

Alle Übungen laufen im Übungsprojekt `claude-code-uebungen`, einem kleinen Java-21-Projekt mit erfundenen Daten. Jede Übung hat einen eigenen Startpunkt als Git-Tag.

```
git clone https://github.com/grigory-consulting/claude-code-uebungen.git
cd claude-code-uebungen
git switch -c meine-lab1 lab1-start
.\mvnw.cmd test
```

So wechseln Sie vor jeder Übung auf ihren Startpunkt, hier als Beispiel für Lab 1. Lab 0 läuft auf `main`.

Unter macOS oder Linux heißt der Build-Befehl `./mvnw test`.

Auf den nächsten Startpunkt wechseln. Sichern Sie zuerst Ihren Stand, sonst bricht der Wechsel an eigenen Dateien wie `CLAUDE.md` oder `.claude/` ab:

```
git add -A
git commit -m "Stand Lab 2"
git switch -c meine-lab3 lab3-start
.\mvnw.cmd clean test
```

Das `clean` gehört dazu: Der Ordner `target` bleibt beim Wechsel liegen. Alte Testklassen aus einer früheren Übung (zum Beispiel aus Lab 5) laufen sonst mit und werden rot, obwohl es ihre Quellen nicht mehr gibt.

Ab `lab3-start` enthält der Startpunkt eine CLAUDE.md, ab `lab8-start` und `lab9-start` Dateien unter `.claude/`. Das sind mögliche Lösungen früherer Übungen, damit alle mit demselben Stand weitermachen.

Bei Aufgaben mit **Tipp vorab** schreiben Sie Ihre Vermutung in den Teams-Chat, bevor Sie Claude starten. Danach vergleichen Sie.

Lab 0 · Setup-Check

Start: `main`

1. `claude --version` und `claude doctor` ausführen.
2. Repository klonen, `.\mvnw.cmd test` ausführen. Der Build muss grün sein.
3. Projekt in IntelliJ öffnen und `claude` im Terminal von IntelliJ starten. Im Vertrauensdialog mit Pfeil nach unten „Yes, I trust this folder“ wählen, die Voreinstellung ist „No, exit“. Fragen Sie Claude zuerst, welche Module das Projekt hat und wie sie voneinander abhängen.
4. Fragen Sie Claude, was die Methode `asText` in `Invoice.java` liefert. Nur fragen, nichts ändern lassen. Mit dem optionalen Plugin „Claude Code [Beta]“ können Sie die Methode vorher markieren; im Transkript erscheint dann eine Zeile wie `⧉ Selected 3 lines from …`.

Fertig, wenn: Der Build grün ist und Claude Ihre Frage zu `asText` beantwortet hat.

Lab 1 · Erste Schritte

Start: `lab1-start`

1. **Tipp vorab:** Sie arbeiten im Modus Accept edits und bitten Claude, `Invoice.asText()` um das Fälligkeitsdatum zu ergänzen und danach `.\mvnw.cmd test` auszuführen. Bei welchem der beiden Schritte fragt Claude Sie?
2. Prüfen Sie Ihre Vermutung. Den Modus wechseln Sie mit `Shift`+`Tab`, die Statuszeile zeigt ihn an. Probieren Sie danach den Modus aus, bei dem Sie sich am unsichersten sind, und schreiben Sie Ihre Beobachtung in den Chat.

3. Geben Sie einen größeren Auftrag: „Stelle alle Ausgaben von `asText()` auf Englisch um und merke die geänderten Dateien mit `git add` vor.“ Holen Sie danach den alten Stand mit `/rewind` zurück, ohne selbst Git zu benutzen. Was zeigt jetzt `git status`, was zeigt der Editor?
4. Schauen Sie mit `/context`, wie viel Kontext belegt ist, bevor und nachdem Claude alle Klassen in `billing/src/main` gelesen hat.

Arbeiten Sie für diese Übung in PowerShell außerhalb von IntelliJ. Im Terminal von IntelliJ empfiehlt die Claude-Code-Doku zu JetBrains den Modus Manual, weil Accept edits und Auto Mode dort auch Konfigurationsdateien der IDE ändern können.

Fertig, wenn: Sie erklären können, was Rewind zurückgeholt hat und was nicht, und warum.

Lab 2 · CLAUDE.md für das Hausframework

Start: `lab2-start`

Auftrag für Claude (in beiden Läufen gleich):

> Implementiere einen Säumniszuschlag: 1 % des rückständigen Betrags je angefangenem Monat nach Fälligkeit, kaufmännisch gerundet. Protokolliere jede Berechnung. Schreibe Tests und führe sie aus.

1. **Tipp vorab:** Welche Regeln aus `docs/team-standards.md` hält Claude ohne CLAUDE.md ein, und woher hätte es sie?
2. Erster Lauf ohne CLAUDE.md. Sehen Sie sich den Diff an: Woher kommt der Prozentsatz, was steht in der Protokollzeile, mit welchem Befehl hat Claude getestet? Sichern Sie den Diff außerhalb des Repos und verwerfen Sie die Änderung:

```
git add -A
git diff --cached --output=..\ohne-claude-md.diff
git reset --hard
```

3. Legen Sie mit `/init` eine CLAUDE.md an und kürzen Sie sie auf das, was Claude nicht aus dem Code sieht: Build- und Testbefehle unter Windows, die Regeln aus `docs/team-standards.md` (per Import), Grenzen der Module.
4. Neue Sitzung, derselbe Auftrag. Vergleichen Sie mit dem ersten Diff.
5. Optional: Legen Sie eine persönliche Vorliebe (zum Beispiel die Antwortsprache) so ab, dass sie in allen Ihren Projekten gilt.

Fertig, wenn: Ihre CLAUDE.md höchstens etwa 40 Zeilen hat, jede Regel im Diff prüfbar ist und Sie benennen können, welche Unterschiede zwischen den beiden Läufen auf die CLAUDE.md zurückgehen.

Lab 3 · Codebasis analysieren

Start: `lab3-start`

1. **Tipp vorab:** An wie vielen Stellen wird auf einer Rechnung mit Rabatt gerundet?
2. Fragen Sie Claude: „Ein Kunde wechselt am 20.09.2026 vom Tarif Basis in den Tarif Komfort. Was steht auf seiner Septemberrechnung? Nenne die Aufrufkette mit Klassen und Methoden.“ Prüfen Sie jede Behauptung im Code (IntelliJ: Aufrufhierarchie, Breakpoint oder ein kleiner Test).
3. Fragen Sie: „Welche Klassen lesen `Configuration`, und welchen Schlüssel liest jede?“ Prüfen Sie die Antwort mit der Suche in IntelliJ.

4. Lassen Sie Claude für einen Tarif, zu dem es noch keinen Test gibt, den Bruttobetrag einer Monatsrechnung ausrechnen. Schreiben Sie dazu selbst einen Test und vergleichen Sie.

Fertig, wenn: Sie für jede Aussage von Claude eine Fundstelle im Code haben oder sie als falsch markiert haben.

Lab 4 · Feature mit Plan Mode

Start: `lab4-start`

Ticket:

> Beginnt ein Vertrag mitten im Monat, soll für diesen ersten Monat nur anteilig abgerechnet werden.

1. Geben Sie Claude das Ticket unverändert im Plan Mode. Lesen Sie den Plan, bevor Sie ihn freigeben: Welche Annahmen trifft er, die nicht im Ticket stehen? Sammeln Sie sie.
2. Entscheiden Sie die offenen Punkte selbst. Schreiben Sie daraus einen Auftrag mit Ziel, Randbedingungen und Akzeptanzkriterien (mit Beispielzahlen).
3. Lassen Sie Claude mit diesem Auftrag neu planen und dann umsetzen.
4. Prüfen Sie den Diff und die neuen Tests gegen Ihre Kriterien. `.\mvnw.cmd test` muss grün sein.
5. Stellen Sie den Reviewauftrag von der Folie „Reviewauftrag“ auf Ihren Diff und prüfen Sie jeden Befund am Code.

Fertig, wenn: Ihre Akzeptanzkriterien jede Annahme festlegen, Tests sie abdecken und kein `[Muss]`-Befund offen ist.

Lab 5 · Legacy modernisieren

Start: `lab5-start`

Ziel: das Modul `legacy` (`PaymentFileReader`, `Payment`, `PaymentReport`) auf modernes Java 21 umbauen: Records, `java.time`, `List` und `Map` statt `Vector` und `Hashtable`, keine `null`-Rückgaben, Geld als `Money`. Für `Money` darf `legacy` eine Abhängigkeit auf `core` bekommen. Das fachliche Verhalten bleibt gleich.

1. Lassen Sie Claude erklären, was der Leser tut und welche Fachregeln darin stecken. Prüfen Sie die Erklärung am Code.
2. Lassen Sie Charakterisierungstests schreiben, die das heutige Verhalten mit der Beispieldatei festhalten. Die Tests laufen gegen den alten Code grün. Committen Sie diesen Stand.
3. Lassen Sie modernisieren, in kleinen Schritten, nach jedem Schritt `.\mvnw.cmd test`. Bei Umbauten der Schnittstelle passen Sie in den Tests nur Aufrufe und Typen an, keine erwarteten Werte.
4. Kürzbar: `docs/architecture.md` im selben Commit nachziehen lassen.

Fertig, wenn: Alle Charakterisierungstests grün sind, kein erwarteter Wert geändert wurde und jede bewusste Verhaltensänderung in der Commit-Nachricht steht.

Lab 6 · Fehler über einen Test

Start: `lab6-start`

Meldung aus dem Fachbereich:

> Ein Kunde zahlt 100,00 € in drei Raten. Im Ratenplan stehen 3 × 33,33 €, es fehlt ein Cent.

1. **Tipp vorab:** Gibt es Beträge und Ratenzahlen, bei denen der Kunde zu viel zahlt?

2. Lassen Sie zuerst einen Test schreiben, der den Fehler reproduziert. Der Test muss rot sein, bevor irgendetwas am Produktionscode geändert wird.
3. Legen Sie fest, welche Rate einen Restcent trägt, und halten Sie die Entscheidung im Test fest.
4. Lassen Sie den Fehler beheben und alle Tests ausführen. Prüfen Sie den Diff Zeile für Zeile und entscheiden Sie bei jedem rot gewordenen Test, ob der Test oder der Code falsch ist.
5. Lassen Sie ein Review Ihres Diffs vorbereiten (Auftrag an Claude oder an einen Subagenten) und gehen Sie die Befunde durch.

Fertig, wenn: Der neue Test erst rot, dann grün war und Sie jede geänderte Zeile in Test und Code begründen können.

Lab 7 · Skill, Slash Command und Hook

Start: `lab7-start`

1. Legen Sie einen Skill `billing-review` an, der Ihre Änderungen gegen `docs/team-standards.md` prüft und die Befunde als Liste ausgibt. Rufen Sie ihn als `/billing-review` auf. Haben Sie den Ordner `.claude/skills/` erst während der Sitzung angelegt, laden Sie die Skills mit `/reload-skills` neu.
2. **Tipp vorab:** In der CLAUDE.md steht, dass eine Änderung erst mit grünem `mvnw test` fertig ist. Sie geben den Auftrag: „Benenne `Money.dividedBy` in `divideBy` um, nur in `Money.java`. Führe keinen Build aus, die Aufrufer passe ich selbst in IntelliJ an.“ Baut Claude trotzdem? Erfahren Sie aus dem Transkript, dass der Build danach rot ist?
3. Prüfen Sie Ihre Vermutung mit genau diesem Auftrag. Danach `.\mvnw.cmd -q compile` im Terminal: Welche Datei ist rot? Nehmen Sie die Änderung mit `git restore core` zurück.
4. Legen Sie einen PostToolUse-Hook an, der nach jeder Änderung an Java-Dateien kompiliert und Fehler an Claude zurückmeldet. Der Hook ist eine einzige Befehlszeile in `.claude/settings.json`, ausgeführt von Git Bash, ohne eigene Skriptdatei. Testen Sie die Zeile in Git Bash von Hand mit einem Beispiel-JSON, bevor Claude sie auslöst.
5. Wiederholen Sie den Auftrag aus Schritt 2 wörtlich. Was meldet der Hook, und was tut Claude daraufhin?

Fertig, wenn: Skill und Hook unter `.claude/` liegen und Sie erklären können, warum die Regel aus der CLAUDE.md in Schritt 3 nicht gegriffen hat und der Hook in Schritt 5 trotzdem lief.

Lab 8 · Sicherheitsregeln

Start: `lab8-start`

1. **Tipp vorab:** Sie sperren `data/real/` für Claude per Deny-Regel. Kommt Claude danach noch an den Inhalt von `data/real/customer-export.csv`, mit dem Lesewerkzeug oder über einen Shell-Befehl? Mit dem optionalen Plugin „Claude Code [Beta]“ können Sie zusätzlich Zeilen der Datei markieren und fragen, ob Claude die Auswahl sieht.
2. Tragen Sie `Read(./data/real/**)` als Deny-Regel in `.claude/settings.json` ein und `Read(./data/**)` als Allow-Regel in `.claude/settings.local.json`. Bitten Sie Claude, die Datei zu lesen, und prüfen Sie Ihre Vermutung aus Schritt 1.
3. Probieren Sie, ob Claude die Datei über einen Shell-Befehl lesen kann, und notieren Sie, welches Werkzeug (Bash oder PowerShell) Claude dafür benutzt.
4. Legen Sie einen PreToolUse-Hook an, der `git commit` blockiert, wenn die zu committenden Änderungen eine IBAN enthalten. Testen Sie ihn mit einer Datei, die eine erfundene IBAN enthält. Die Meldung darf die IBAN nicht im Klartext zeigen. Prüfen Sie zum Schluss, dass ein Commit ohne IBAN durchgeht. Denken Sie an beide Shells.

Fertig, wenn: Claude die Datei mit dem Lesewerkzeug nicht lesen kann, Sie das Ergebnis für die Shell notiert haben und der Commit mit IBAN mit einer Begründung blockiert wird.

Lab 9 · Praxisworkshop

Start: `lab9-start`

Ticket:

> Kündigt ein Kunde zu einem Tag mitten im Monat, ist dieser Monat schon voll berechnet. Er erhält für die restlichen Tage des Monats eine Gutschrift.

Pflicht:

1. Kontext prüfen (ist die CLAUDE.md aktuell?), Ticket im Plan Mode planen lassen, offene Fragen entscheiden, Auftrag mit Akzeptanzkriterien schreiben.
2. Umsetzen lassen, Tests schreiben lassen, `.\mvnw.cmd test` grün.
3. Auf einem eigenen Branch committen, Commit-Message prüfen. Zeigen Sie mit `/hooks`, dass der IBAN-Hook aktiv ist, oder lassen Sie ihn einmal einen Commit mit erfundener IBAN ablehnen.

Kür:

4. PR-Beschreibung erzeugen lassen (Ziel, Änderungen, Testnachweis, offene Punkte).
5. `/billing-review lab9-start` und Diff lesen. Prüfen Sie, ob Claude Texte aus dem Repository als Anweisung behandelt hat.

Fertig, wenn: Branch und Commit stehen und Sie jede Zeile im Diff erklären können.